

openSUSE Leap Server as an Ideal Platform for Enterprise Data Lakehouse

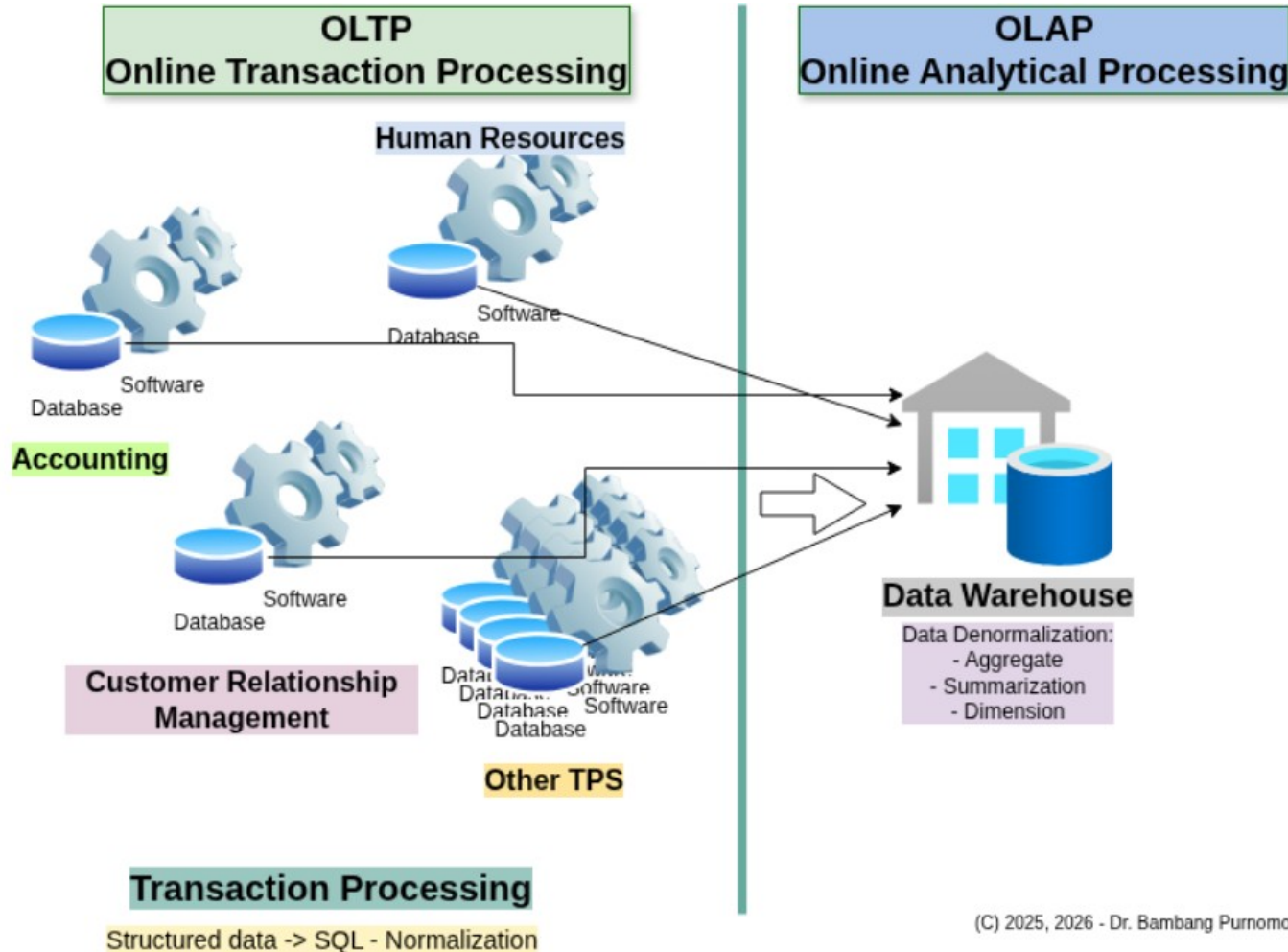
Bambang Purnomosidi D. P.

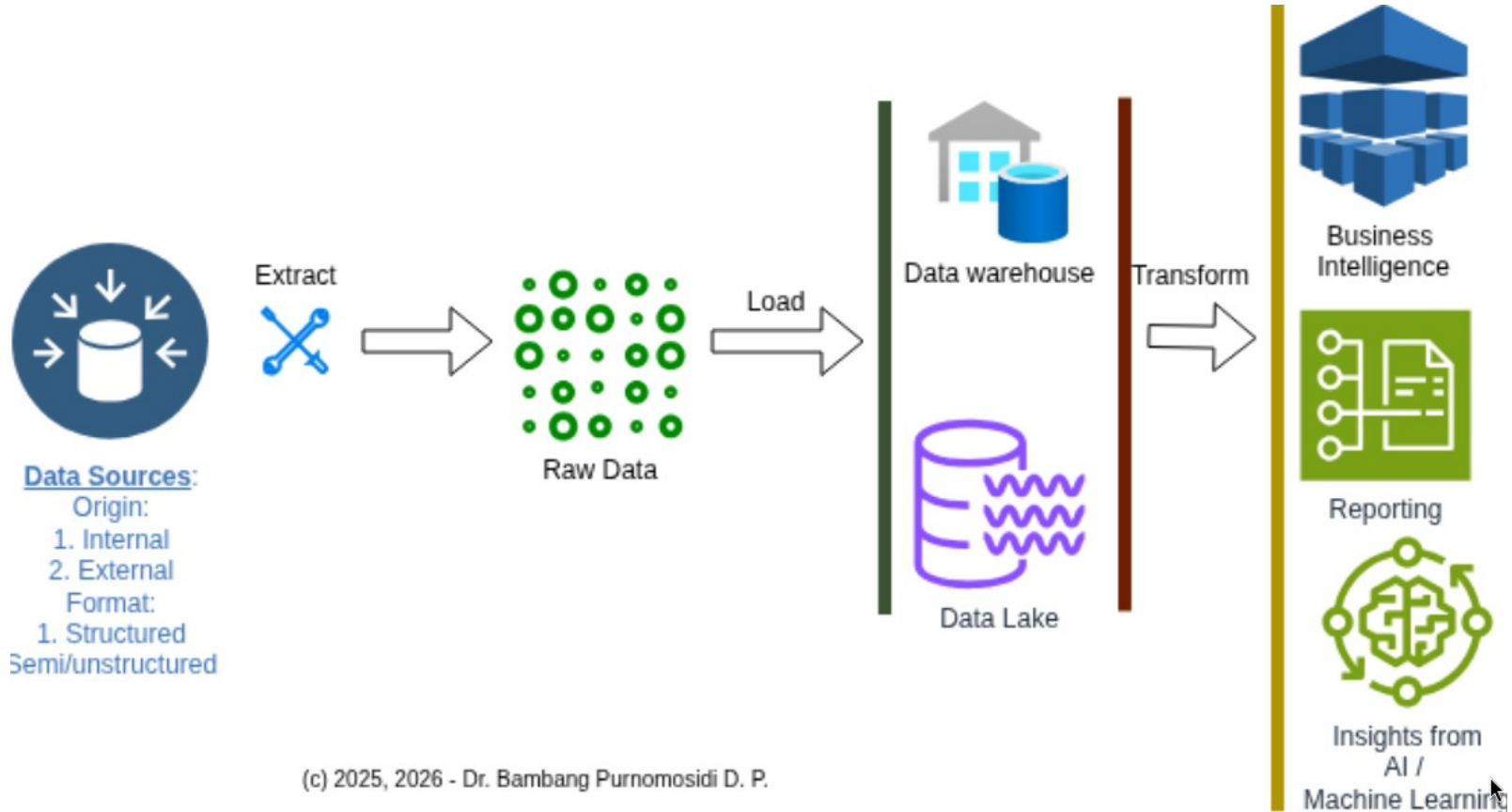
\$ whoami

- Lecturer and Researcher at Universitas Teknologi Digital Indonesia (<https://www.utdi.ac.id>)
- Head of all human at PT Neo Akselerasi Indonesia (<https://neo-x.id>)
- <https://linktr.ee/bpdp>

A Basic Concept of Data Lakehouse

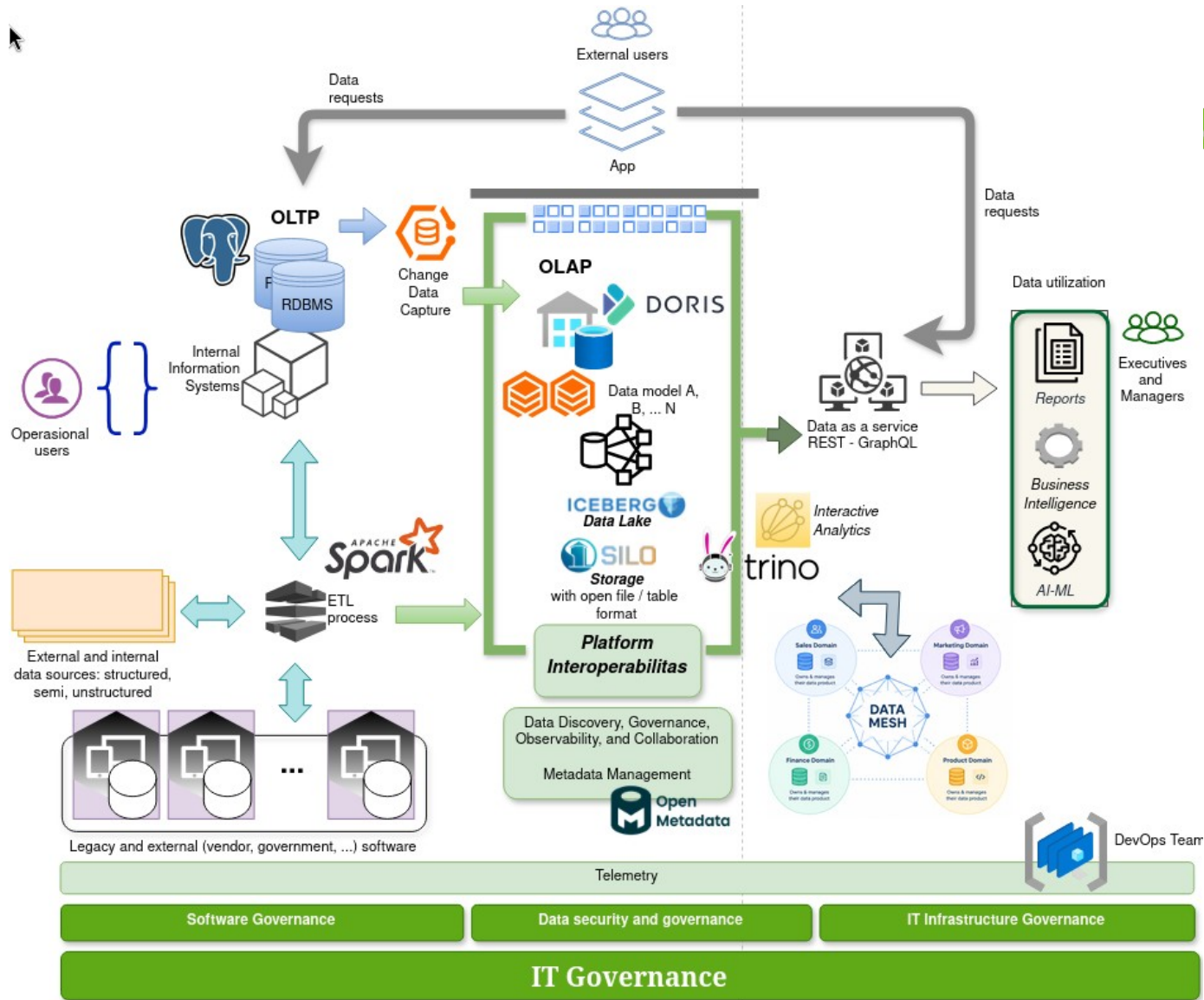
- Facts → Data
- Data → Database
- Database → Data Warehouse
- Semi-structured / Unstructured Data → Data Lake
- Data Warehouse + Data Lake → **Data Lakehouse**





Architecture of Enterprise Data Lakehouse

- Enterprise Data Lakehouse (EDL) is considered as **wicked problem**:
 - No single correct way
 - No stopping rule
 - No right or wrong answers
 - No trial runs
- So, this is my attempt to define and implement EDL. **Warning:** very opinionated.



openSUSE Leap Server for EDL

- An OS is a very important underlying component in EDL.
- An OS should provides: (1) stable and robust platform for heavy data workloads. (2) tools support.
- OpenSUSE Leap Server combines source from SLES (SUSE Linux Enterprise Server) and community developments. SLES provides stable and robust platform while community developments provide low cost and extended software components. It doesn't use rolling release model that can break unexpectedly.
- Extensive Tools support: zypper, cockpit, multi kernel version, server software, development tools, you name it.
- End-of-life: 2 years (2025 – 2027) for openSUSE Leap 16.x. but zypper provides smooth upgrades to next major version, so basically, no end of life for openSUSE Leap Server 😊
- openSUSE Leap packages are thoroughly tested through automated QA processes, community testing, and an enterprise foundation shared with SUSE Linux Enterprise (SLE). Packages are built and integrated through the openSUSE Build Service, which performs automated checks and dependency validations.

Multi Environment in EDL

- Some software requires the same development tools / SDK / runtime but may need different version. Use environment-package manager. Avoid SDK and libraries packaged by OS.
- Example: Apache Cassandra needs JDK 11/17 while Trino needs JDK 25. Python usage needs complex multi version / multi environment for many tools needed in EDL.
- For Java: **SDKMAN** 
- For Python: **uv** 
- For Node.js: **fnm** 

Thank You



open**SUSE**®
Asia Summit 2026